



Universität
Zürich^{UZH}

Department of Chemistry

DSA103 Advanced Chemical Data Science

Lecture 1: Computational Thinking

Prof. Meredith C. Schuman

meredithchristine.schuman@uzh.ch

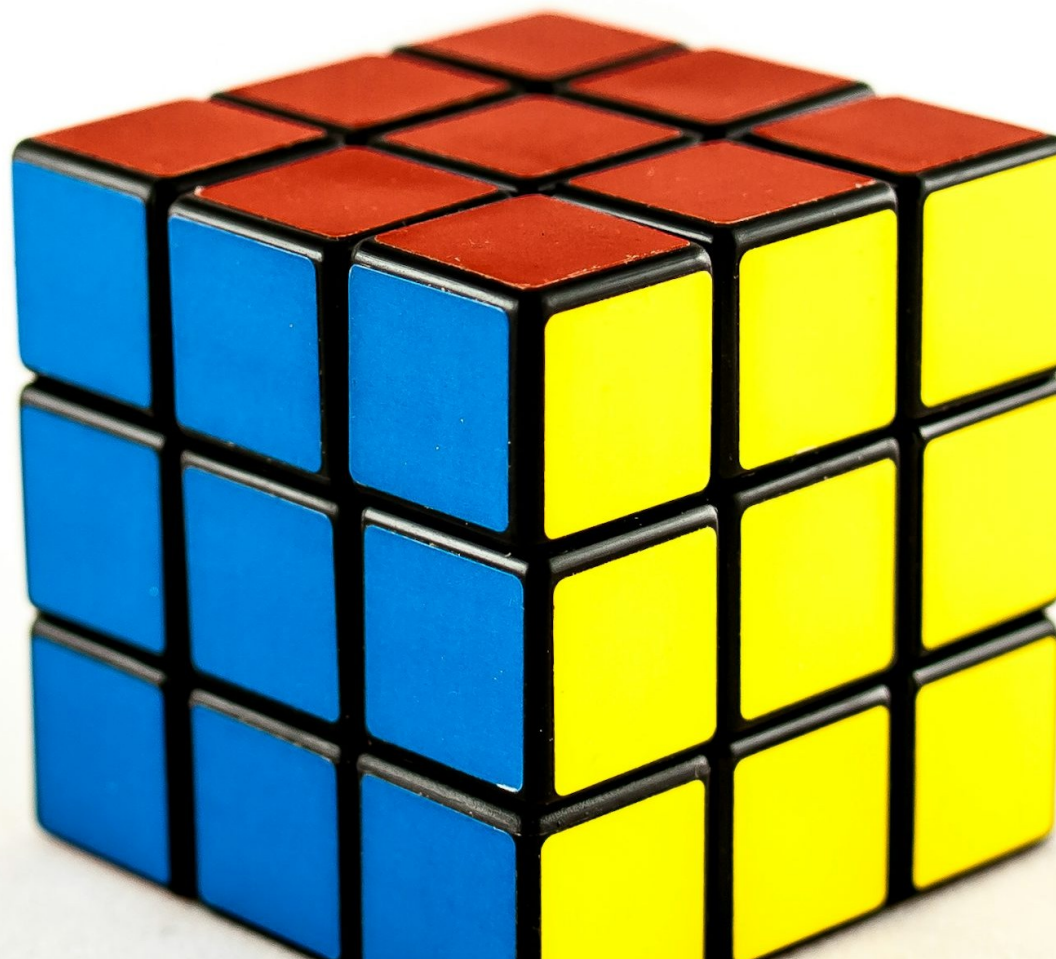


Photo by [William Warby](#) on [Unsplash](#)

Who are we?



Prof. Meredith C. Schuman

Spatial Genetics (for ecology!)

Departments of Geography
and Chemistry



Dimitrios Xynos, PhD Candidate

Biomacromolecular NMR
Spectroscopy

Department of Chemistry



Dr. Johannes Schörgenhumer

High-Throughput
Experimentation Lab (HTEL)

Department of Chemistry



Dr. Marvin Alberts

Wrote exercises



Dr. Alexander Steppke

Helped with solutions

What are we doing here? (DSA103 overview)

Sessions (2/week, Wed and Fri)

Homework assignments will be introduced and reviewed on Fridays (even slots) and there will be the opportunity to discuss and get input on homework assignments on Wednesdays (odd slots).

1. Intro to course and to computational thinking
2. Computational thinking with Python
3. Python review
4. Recap, Q&A, and practice
5. Version control
6. Applying version control for collaboration
7. Exploratory data analysis
8. Finding and evaluating data
9. Generating data for specific purposes
10. Recap, Q&A, and intro to FAIR and CARE frameworks
11. Exploring life science data 1: Data types
12. Exploring life science data 2: Data structures
13. Exploring life science data 3: Data analytics
14. Recap, Q&A
15. Data wrangling without loss
16. Data wrangling with selective changes (outliers & other issues)
17. Putting it together: Small project introduction
18. Putting it together: Small project work
19. Lies, damn lies, and statistics
20. Data visualization: Honest vs. dishonest, useful vs. obfuscating
21. Data visualization continued
22. Recap, Q&A, intro to HPC
23. Outlook: Machine learning
- Putting it together: Small project finalization and submission
25. Outlook: Vibe coding and agentic coding
26. Mock exam
27. Recap, Q&A, feedback round
28. Exam

Interactive course with in-class activities

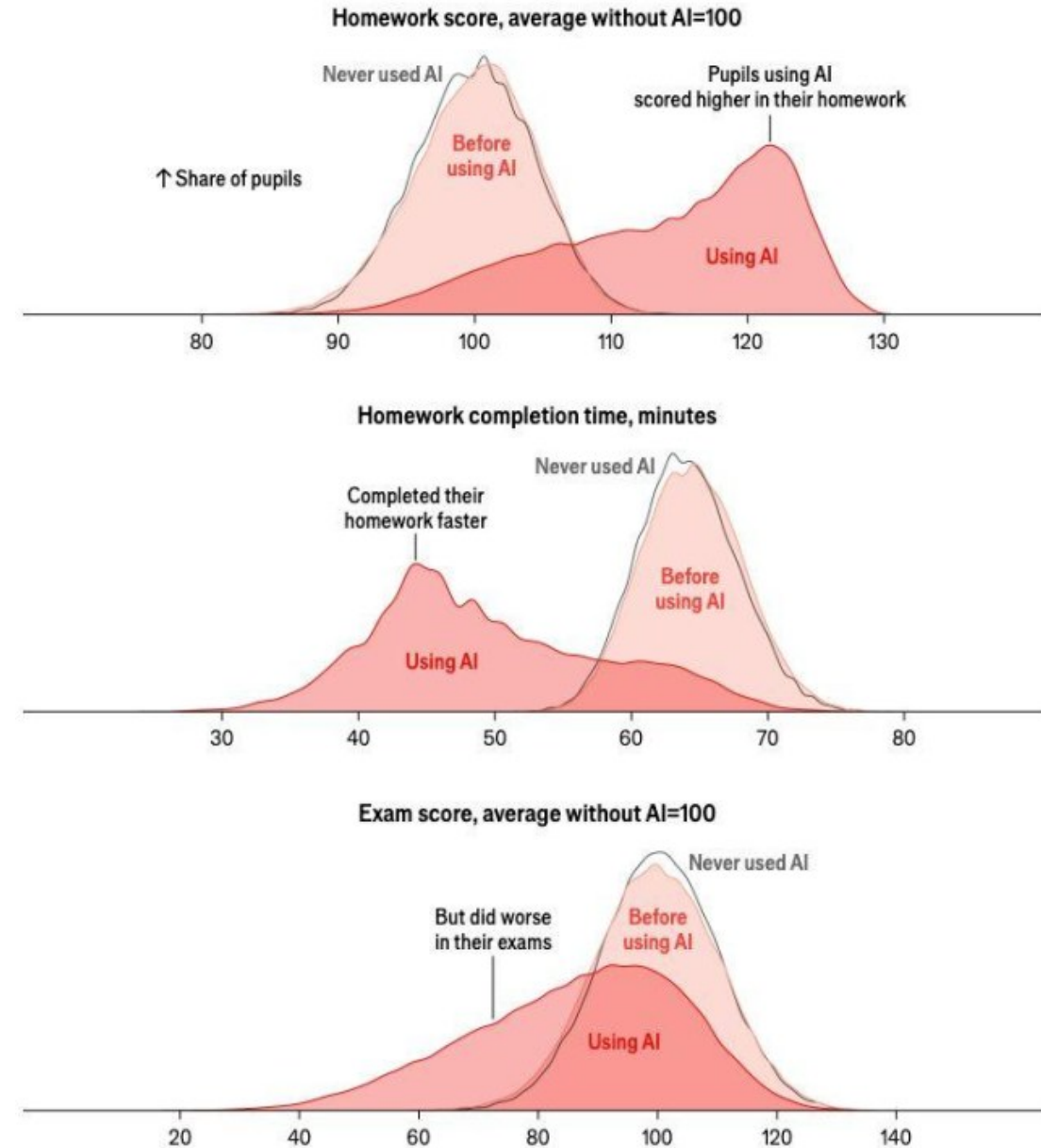
To learn the material, you will need to actively engage with it and **spend time** on the weekly exercises.

We provide a podcast due to course scheduling conflicts and for review.

We recommend active, in-person participation whenever possible, and encourage you to approach us and your classmates.

AI ate my homework

China*, effect of using AI, January 2023-June 2025



Course materials on GitHub

Please fork and clone the repository by Friday. Come with any questions!

(See README; you'll need a free GitHub account.)

[Link to helpful GitHub tutorials](https://github.com/mcschuman/ChemicalDataScience103)

<https://github.com/mcschuman/ChemicalDataScience103>

The screenshot shows the GitHub repository page for 'ChemicalDataScience103'. At the top, the repository name is displayed with a 'Public' badge. Below this, there are buttons for 'main' branch, '1 Branch', and '0 Tags'. A search bar labeled 'Go to file' is present, along with 'Add file' and 'Code' buttons. The commit history section shows a single commit by 'mcschuman' with the message 'chore: initial commit', committed 14 minutes ago. Below the commit history, a list of files is shown: 'assignments', 'lectures', '.gitignore', 'README.md', 'pyproject.toml', and 'uv.lock'. The 'README' file is selected, and its content is displayed below. The README title is 'DSA103'. The content of the README includes a description of the repository, a list of instructors (Prof. Meredith C. Schuman and Dr. Johannes Schörghöhner), and a teaching assistant (Dimitrios Xynos).

ChemicalDataScience103 Public

main 1 Branch 0 Tags

Go to file Add file Code

mcschuman chore: initial commit 624ecf0 · 14 minutes ago 2 Commits

assignments	chore: initial commit	14 minutes ago
lectures	chore: initial commit	14 minutes ago
.gitignore	started	32 minutes ago
README.md	chore: initial commit	14 minutes ago
pyproject.toml	chore: initial commit	14 minutes ago
uv.lock	chore: initial commit	14 minutes ago

README

DSA103

Repository for the course DSA103 Advanced Chemical Data Science. All course material will be shared here, whereas OLAT will only be used for all course-related communication.

Instructors and persons responsible for the course: Prof. Meredith C. Schuman (meredithchristine.schuman@uzh.ch), Dr. Johannes Schörghöhner (johannes.schoergenhumer@chem.uzh.ch)

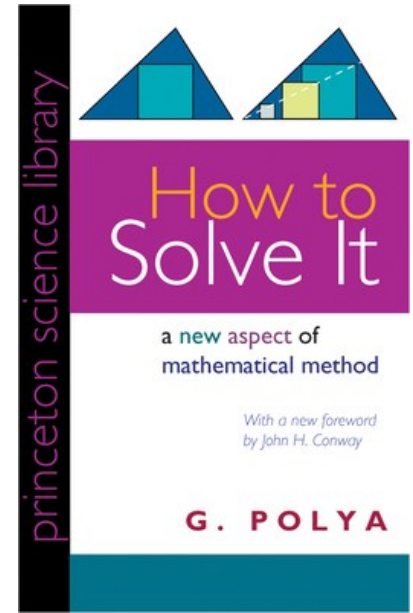
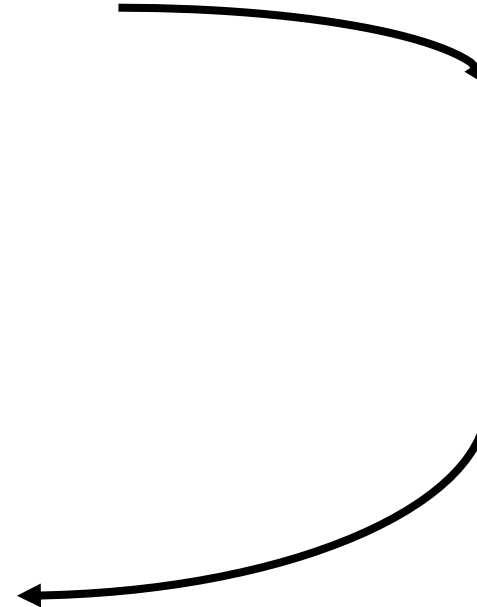
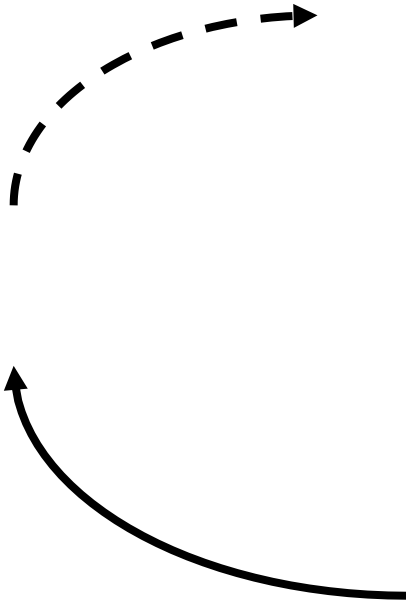
Teaching assistant: Dimitrios Xynos (dimitrios.xynos@chem.uzh.ch)

Prerequisites

What is computational thinking?

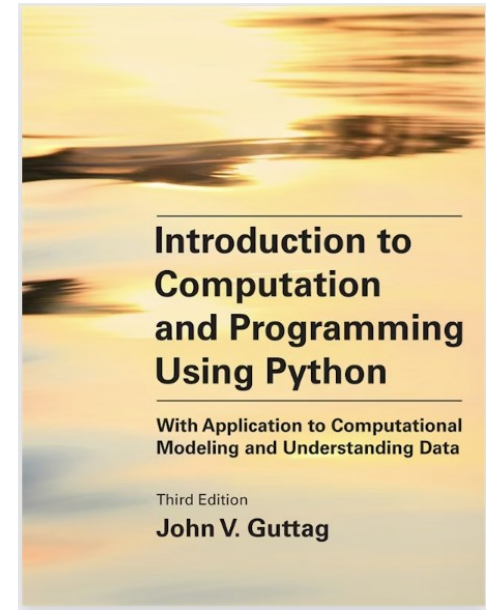
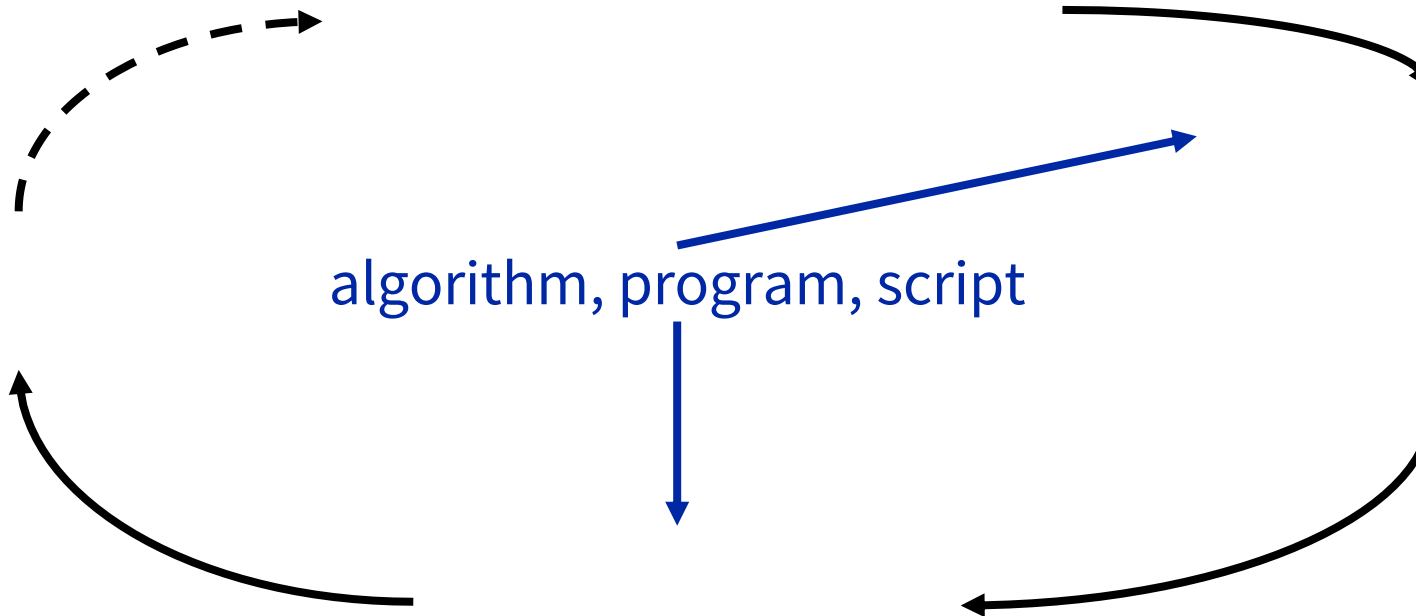
What is computational thinking?

You want to solve a problem using a computer.
What do you do?



What is computational thinking?

You want to solve a problem using a computer.
What do you do?

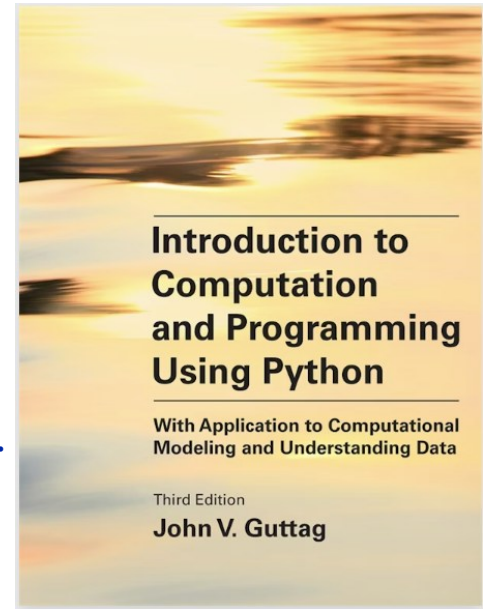


What is computational thinking?

You want to solve a problem using a computer.
What do you do?

It's not enough to show that there is a solution.
You must compute the answer.

Understand that computers
calculate differently than you do.



algorithm, program, script

It's important to robustly
compute a correct answer,
not to find an elegant solution.

You are not limited to what
you can write down.

Computational thinking: components

algorithm ~ recipe

- **pseudocode** ~ “add butter”
- **code** ~ “add 100 g of softened butter to the bowl with the other wet ingredients”

Someone (human, computer) needs to reproduce the procedure according to your instructions.

- good instructions → given the ingredients, you get lovely cupcakes / running code / working synthesis



Computational thinking: components

algorithm ~ recipe

- **pseudocode** ~ “add butter”
- **code** ~ “add 100 g of softened butter to the bowl with the other wet ingredients”

Someone (human, computer) needs to reproduce the procedure according to your instructions.

- good instructions → given the ingredients, you get lovely cupcakes / running code / working synthesis
- bad instructions → not reproducible, context-dependent, poor yield, ...
- ambiguous instructions → not the intended procedure, wrong product



Photo by Brooke Lark on Unsplash



Created with Midjourney

Computational thinking: components

What you find “obvious” may require context & specificity for an algorithm to achieve the desired outcome.

“A photo of cupcakes”



Photo by [Brooke Lark](#) on [Unsplash](#)

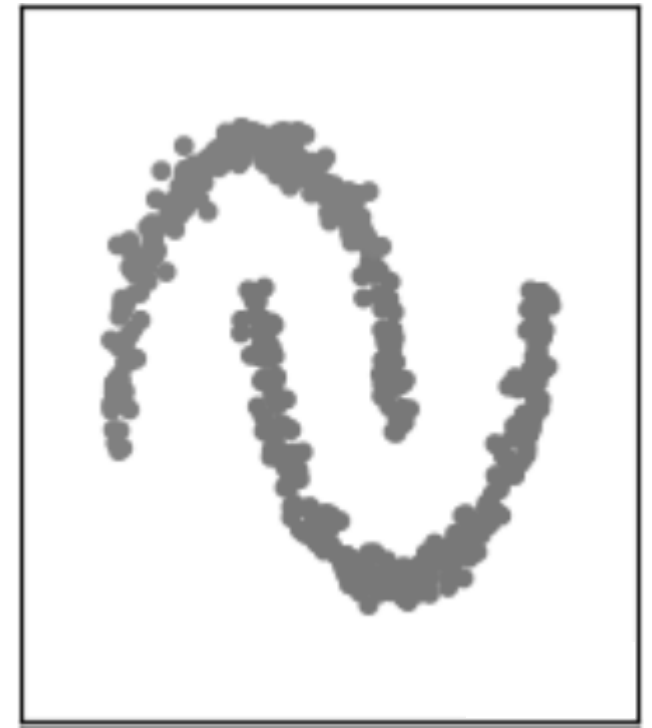
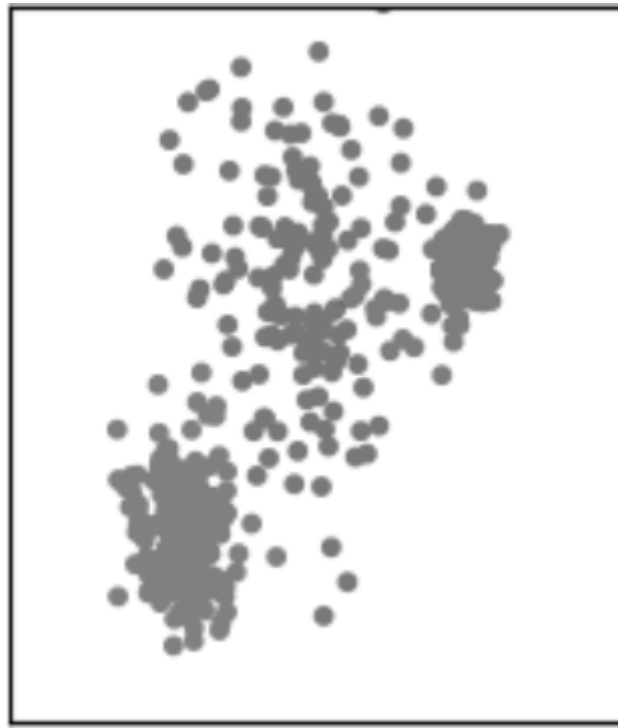
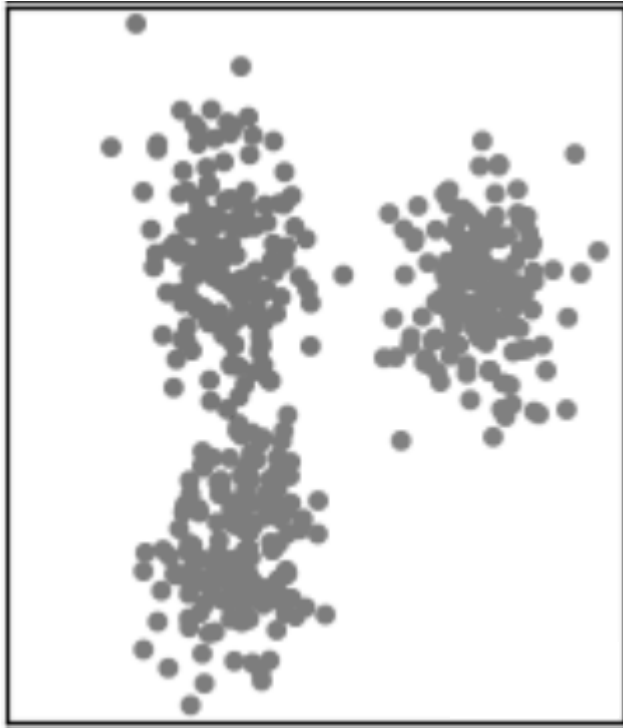


Created with Midjourney

Computational thinking: components

What you find “obvious” may require context & specificity for an algorithm to achieve the desired outcome.

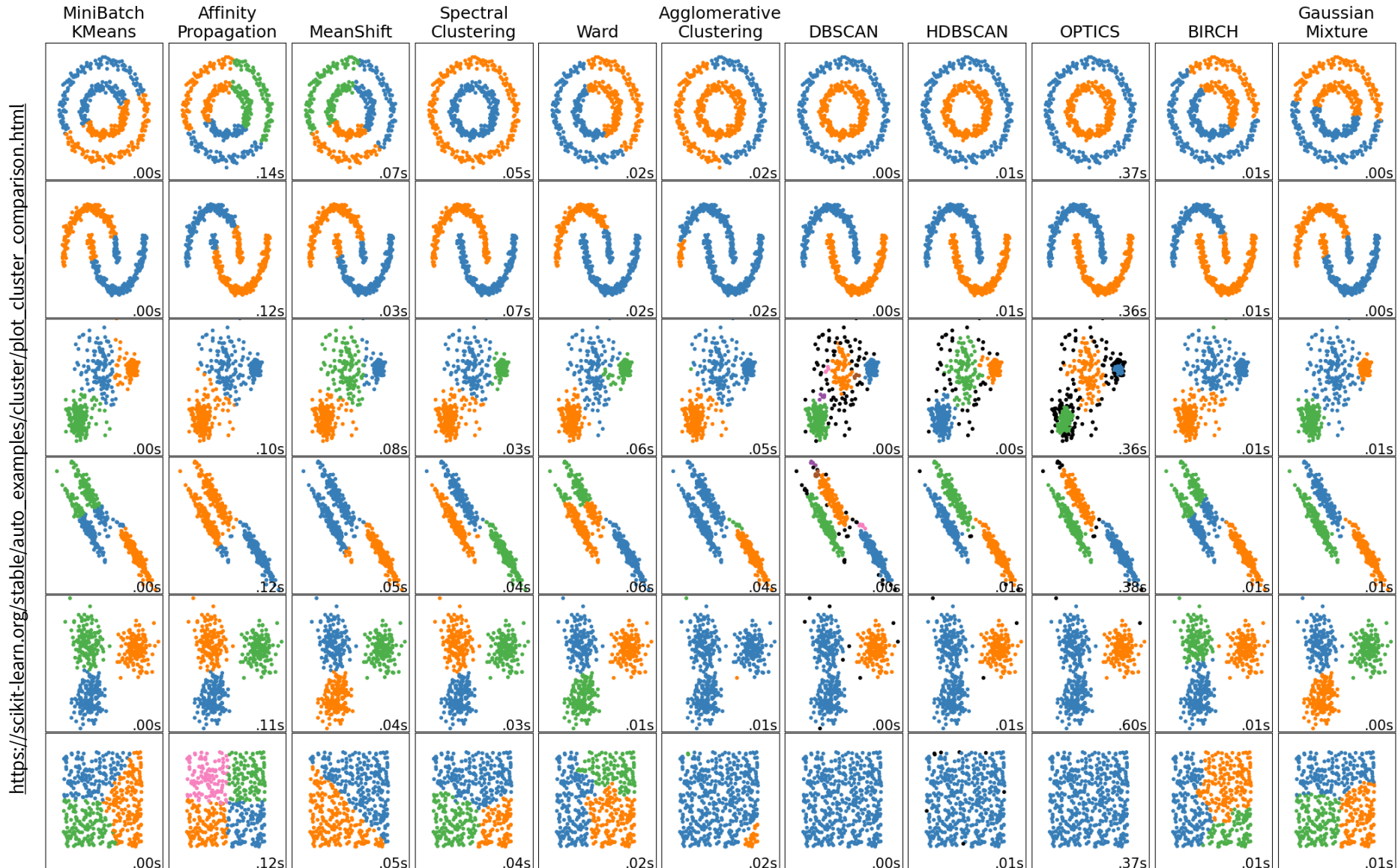
How would you define clusters in these images?



https://scikit-learn.org/stable/auto_examples/cluster/plot_cluster_comparison.html

Computational thinking: components

What you find “obvious” may require context & specificity for an algorithm to achieve the desired outcome.



Computational thinking and data science are closely related.

computational thinking

design algorithms to accomplish a task

~

data science

adopt appropriate algorithms for the data & task
good grasp of statistics required



Computational thinking and data science are closely related.

computational thinking

design algorithms to accomplish a task

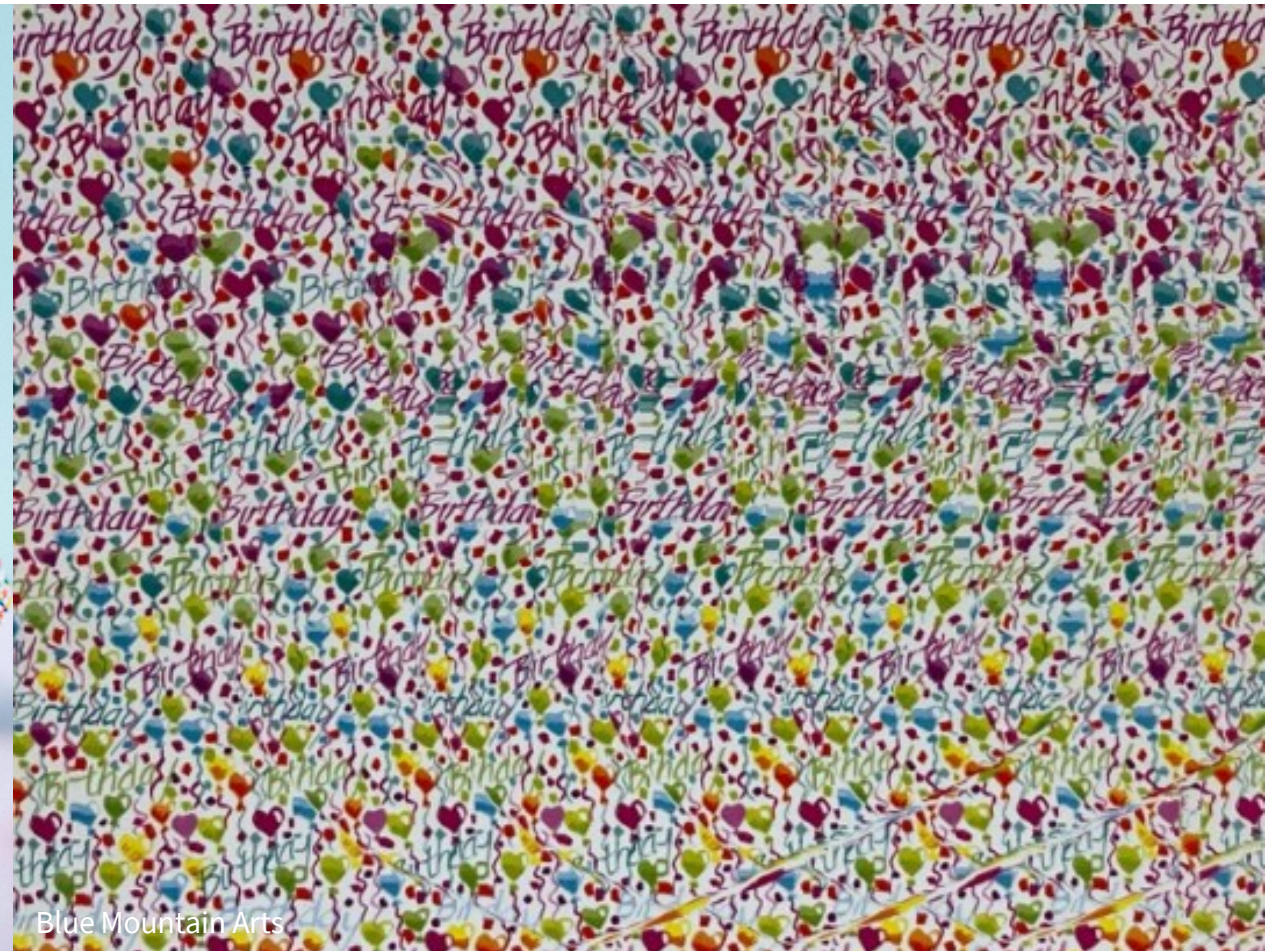
~

data science

adopt appropriate algorithms for the data & task

good grasp of statistics required

core skills: data curation, handling, interpretation



Some examples to warm up to computational thinking

- Think about the **problem statement**
- Write in **pseudocode**: step-by-step description of the algorithm in a code-like structure using plain English (not Python)

Example 1

How would you calculate the mean reaction rates for these substances given this input, using a computer?

Substance	Temperature (°C)	Reaction rate
a	23.4	12.1 mol/h
a	23.5	11.0 mol/h
a	23.1	10.8 mol/h
a	23.6	12.2 mol/h
a	23.8	12.6 mol/h
b	22.1	12.1 mol/h
b	22.5	11.0 mol/h
b	22.7	10.8 mol/h
b	22.1	12.2 mol/h
b	22.8	12.6 mol/h
c	22.8	10.7 mol/h
c	22.8	10.6 mol/h
c	22.8	10.6 mol/h
b	22.8	12.6 mol/h

Considerations:

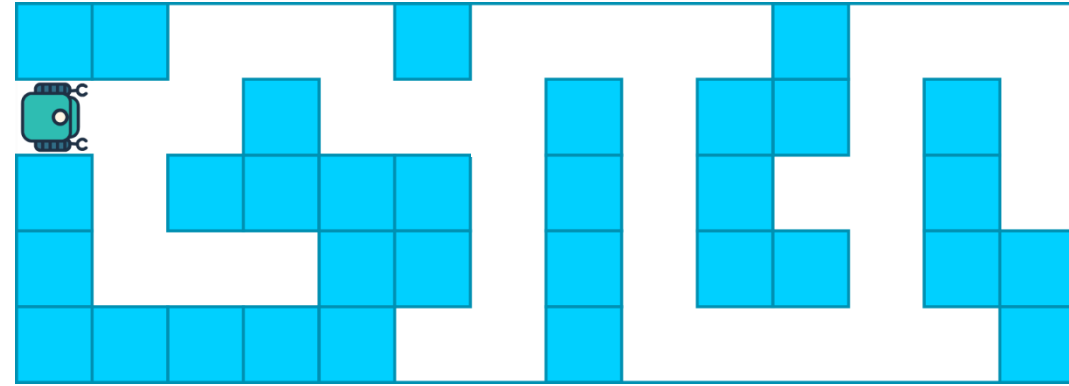
- Exact problem statement
- Order (b)
- Format (mol/h)

Example 2

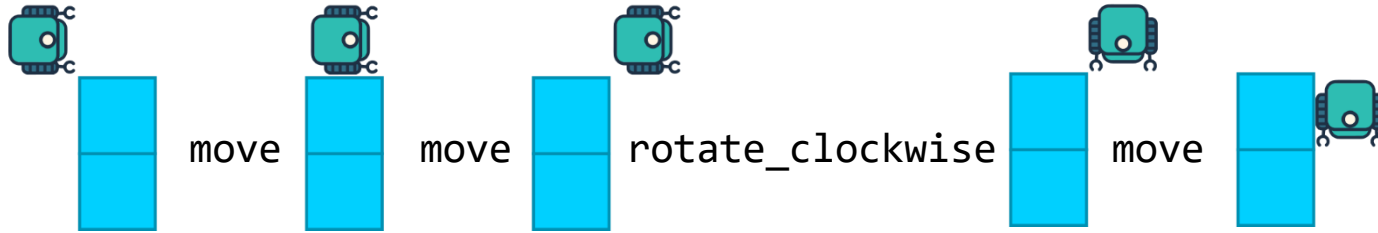
A simple navigational problem: program a robot to move around obstacles and navigate out of a maze.

You have only two commands:

- move
- rotate_clockwise



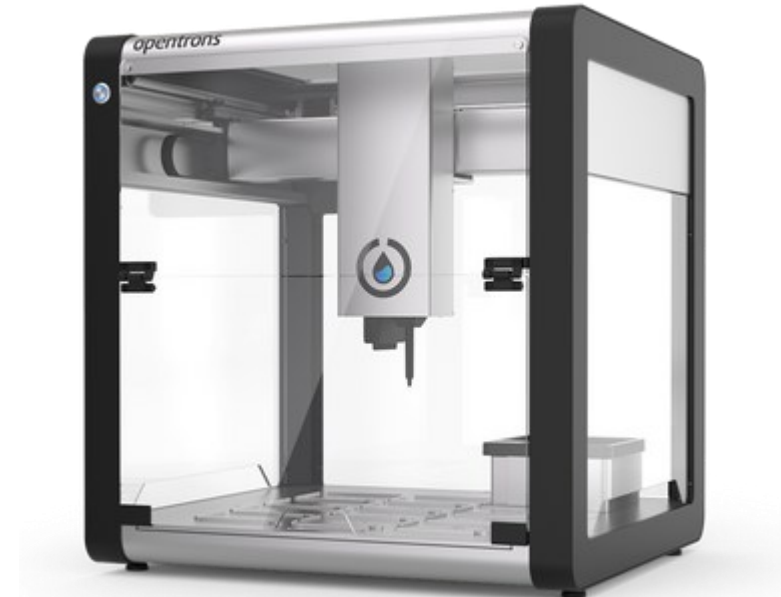
Example



1. Write a program in the above pseudocode to navigate the maze. The robot shows your start & orientation.
2. Afterward you will exchange programs with your neighbor and be each others' "robots". Will your robot make it out without crashing?

Solution

This is similar to **G-code**, which controls robots, milling machines, industrial machines, 3D printers, pipetting robots...



<https://opentrons.com/products/ot-2-robot>

“Instead of electrical impulses sent from a brain, the OT-2/OT-2R uses something called ‘G-code.’”

<https://support.opentrons.com/s/article/Introduction-to-Opentrons-GitHub>

Solution



https://youtu.be/NFrpVqvy0TM?si=Ow8l4gPi_TlVdh1C

“Instead of electrical impulses sent from a brain, the OT-2/OT-2R uses something called ‘G-code.’”

<https://support.opentrons.com/s/article/Introduction-to-Opentrons-GitHub>

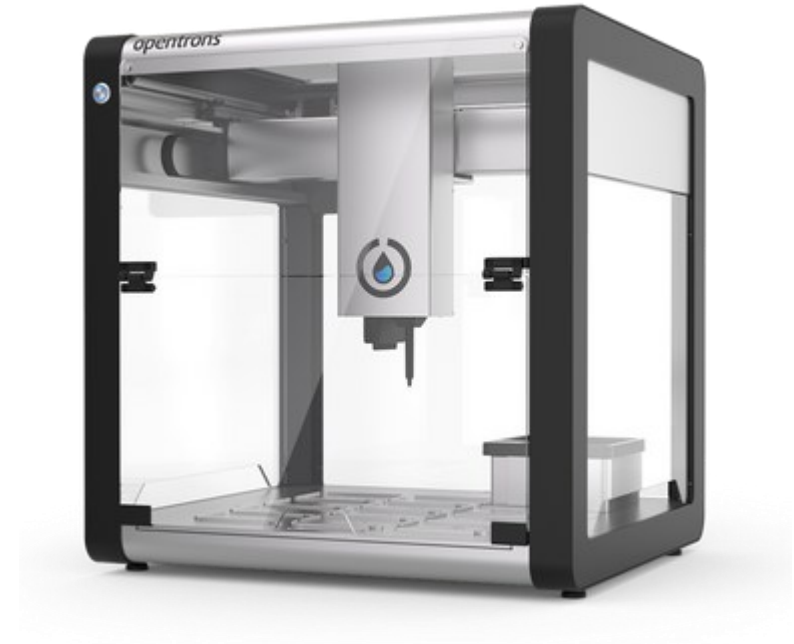
What happens when you make mistakes in G-code?



https://www.youtube.com/watch?v=j0ME_3RwWQ

I didn't find any videos of pipetting robots fails online ;) ... can you?

Solution if you navigate:



<https://opentrons.com/products/ot-2-robot>

But if the robot should navigate...

In the last example you were the navigator. What if the robot should navigate?

Can you derive an algorithm in pseudocode using the following robot capabilities?

- move
- rotate_clockwise
- look_ahead

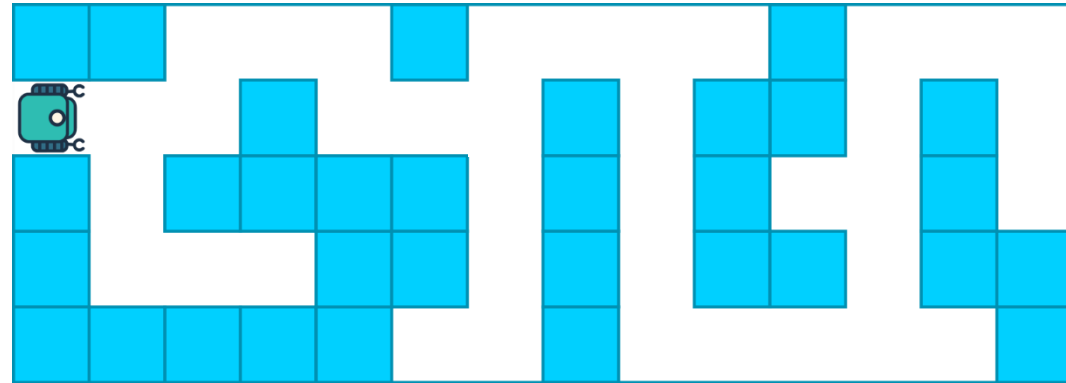
Conditions, functions, etc. are allowed!

Examples

```
if look_ahead is True then  
    rotate_clockwise
```

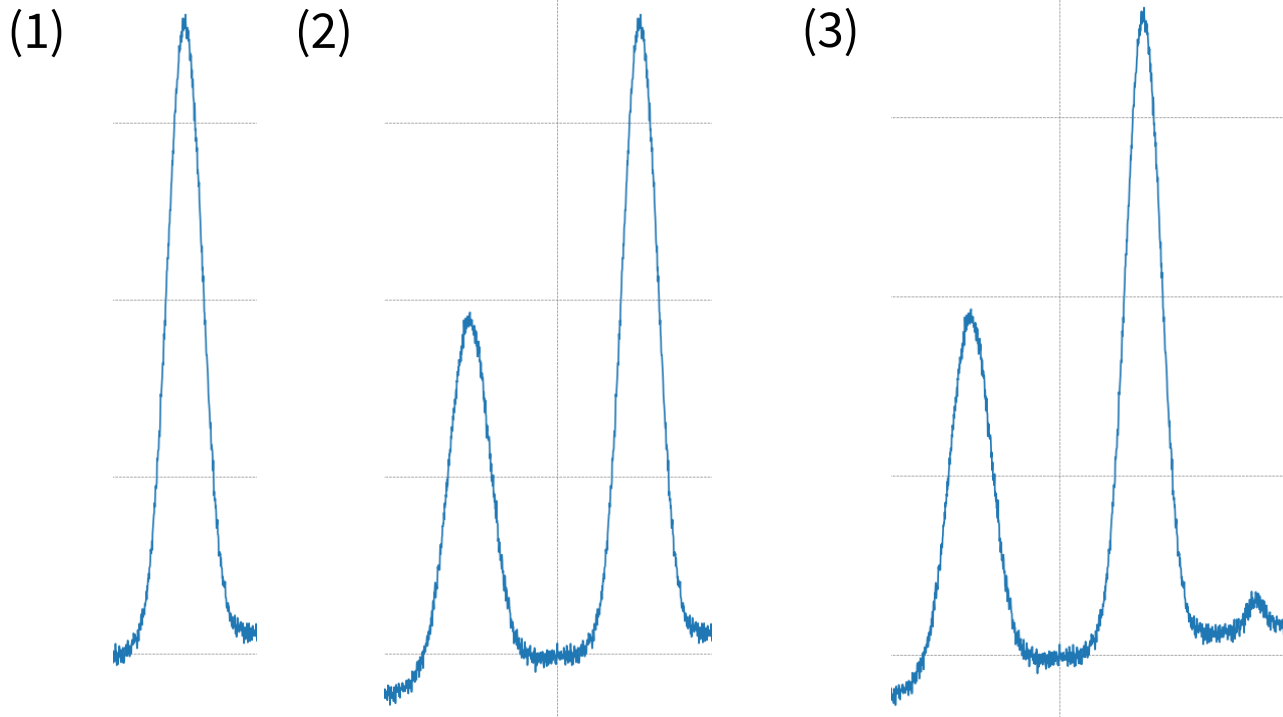
```
function rotate_left  
    rotate_clockwise; rotate_clockwise; rotate_clockwise;
```

This is your homework assignment.
Bring the handwritten solution with you on Friday!



Today's last example: peak-finding (a common problem in data science)

Can you formulate a peak-finding algorithm in pseudocode? What strategy could you take?
First discuss in small groups, then we'll discuss together.

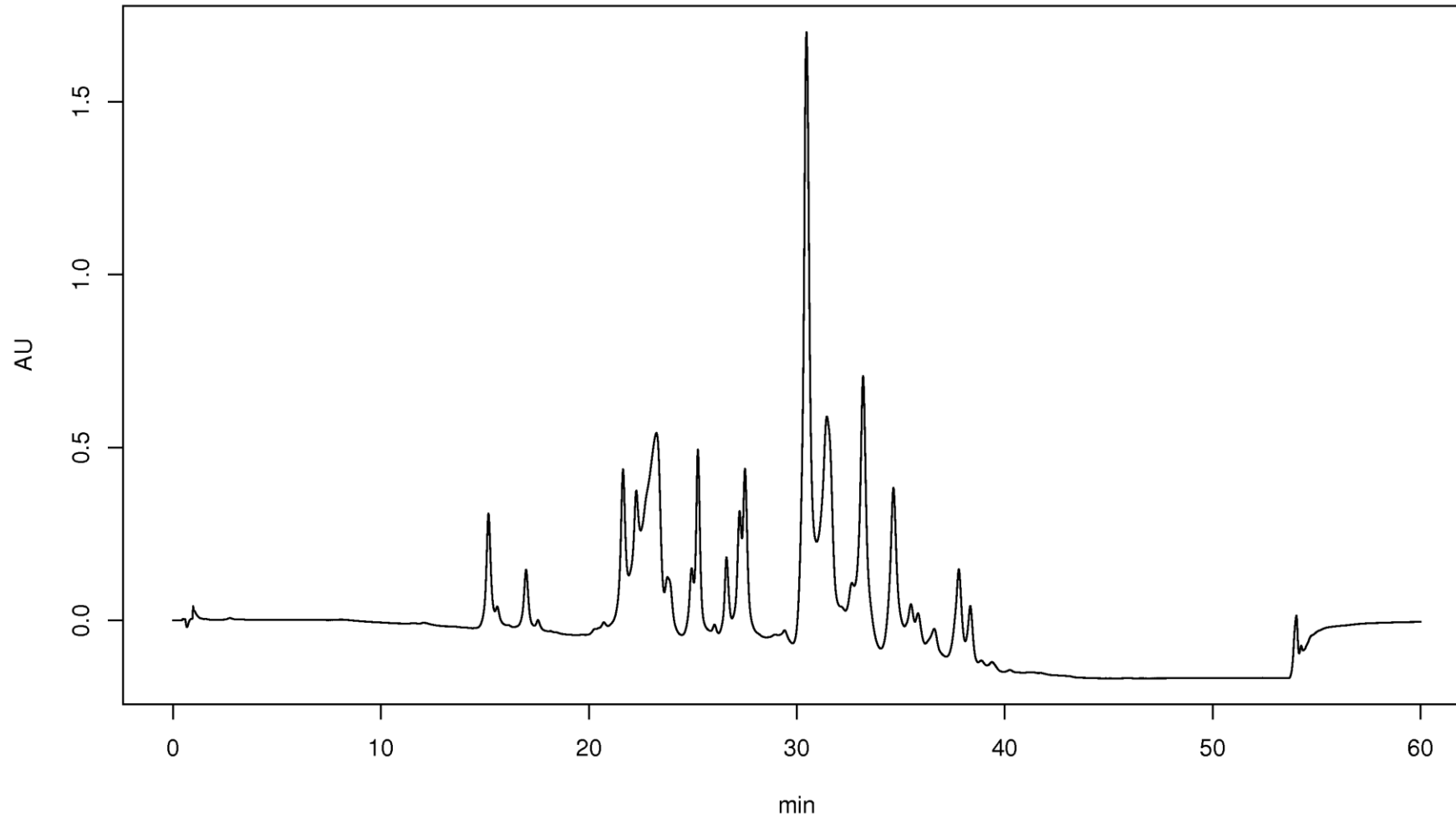


Considerations:

- What is a peak (analytical, computational)
- What is the baseline (analytical, computational)
- Concrete algorithms and trade-offs

Today's last example: peak-finding (a common problem in data science)

Not trivial!



See e.g. MIT 6.006 algorithmic thinking, peak finding ([slides](#) or [video](#)); a [comparison of algorithms used for MS data](#) (older but still useful); and this [StackOverflow post](#) about a [simple option in SciPy](#).

Remember your homework – we'll start with this on Friday!

In the last example you were the navigator. What if the robot should navigate?

Can you derive an algorithm in pseudocode using the following robot capabilities?

- move
- rotate_clockwise
- look_ahead

Conditions, functions, etc. are allowed!

Examples

```
if look_ahead is True then  
    rotate_clockwise
```

```
function rotate_left  
    rotate_clockwise; rotate_clockwise; rotate_clockwise;
```

This is your homework assignment.
Bring the handwritten solution with you on Friday!

